Email services

Some slides are adapted from the slides accompanying the Kurose-Ross textbook

Introductory concepts

Electronic mail is the oldest service offered over the Internet and it is still a very pervasive service and a "must have" (e.g., email accounts required by many services)

Over half of the world population uses email and billions of messages are exchanged daily

Why is it so successful?

- Asynchronous
- Fast
- Easy to use
- Cheap
- Ubiquitous

Many diverse sophisticated security threats affect email services which are often used for blended attacks (e.g., phishing, scam)

Technical details

Two perspectives of email services
- Sending, transfer & delivery of mail messages
- Mail message access

Four independent processes
- Sending/submitting process
- Transferring process
- Delivering process
- Receiving process

Technological infrastructure
- User devices
- Mail servers, i.e., intermediaries between senders and recipients of email messages
- Software agents implementing the service
- Protocol(s) for the communication between software agents

Main components

composition/ submission

transfer/ delivery

access

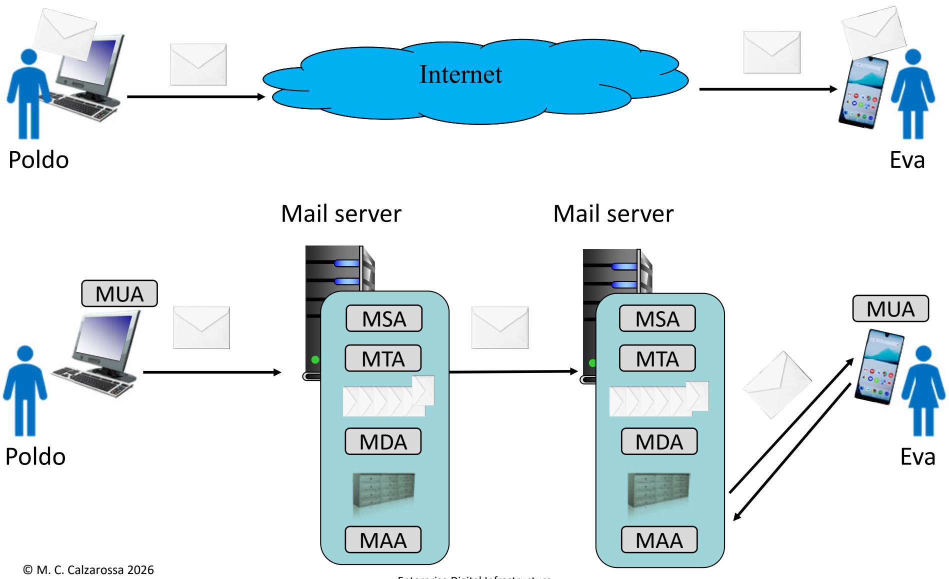
Software agents:

- Mail User Agents running on user devices
- Mail Submission Agents running on Mail Servers
- Mail Transfer Agents running on Mail Servers
- Mail Delivery Agents running on Mail Servers
- Mail Access Agents running on Mail Servers

Storage components on Mail Servers:

- Mailboxes that store email messages of individual users
- Mail store, i.e., *permanent* storage area that stores user mailboxes
- Mail queue, i.e., *temporary* spool area that stores email messages *in transit*

Logical vs physical view



Implementation

Agents are implemented as network applications based on client/server model

- clients request the service (i.e., submit & transfer mail messages)
- servers provide the service (i.e., receive & deliver mail messages)

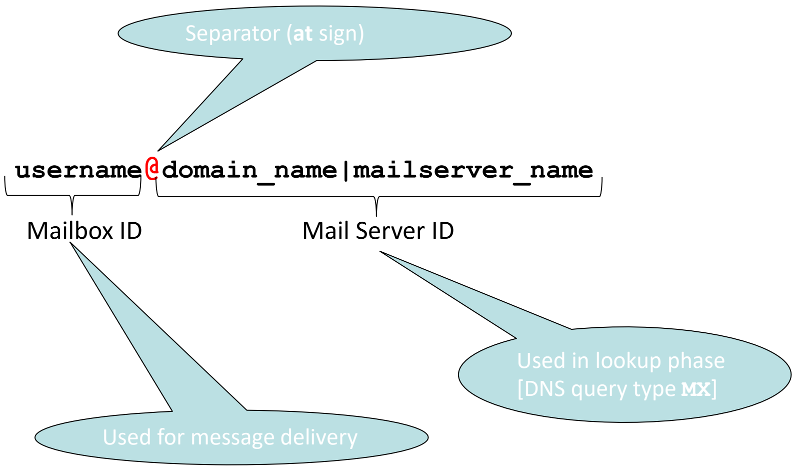
Client and server processes share a common protocol (at the application layer) to communicate between each other over the network

Simple Mail Transfer Protocol (SMTP) is a standard protocol whose objective is to transfer mail reliably and efficiently [RFC 5321 "Simple Mail Transfer Protocol" published in 2008; first RFC published in 1982]

SMTP relies on TCP as transport protocol and standard port number 25; ports 465 or 587 used for mail submission (TLS or STARTTLS), cleartext considered obsolete (RFC 8314 "Cleartext Considered Obsolete: Use of Transport Layer Security (TLS) for Email Submission and Access)" published in 2018")

Reliability is implemented using notification mechanisms

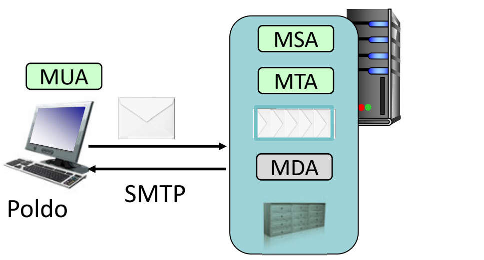
Email address



Detailed example

Poldo wants to send Eva an email message

1. He opens an MUA (or a browser) on his device, specifies Eva's email address (**`eva@mail.com`**) and the mail subject, writes the message and presses "Send"
2. The MUA transfers the message to a "known/trusted" Mail Server (i.e., Local Mail Server):
   - lookup phase & TCP connection
   - SMTP session



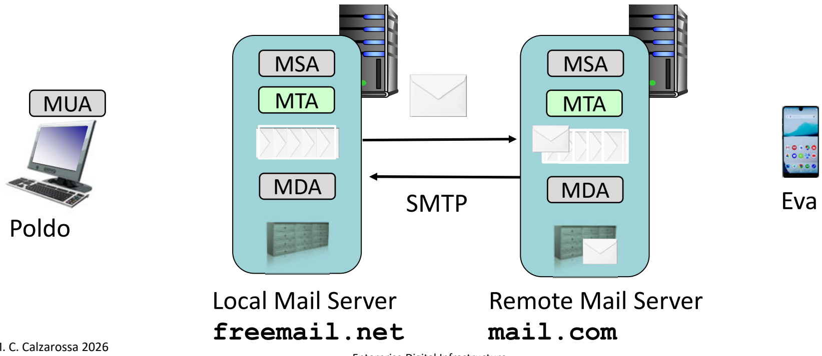
Local Mail Server **`freemail.net`**

No direct connection between MUA and recipient Mail Server

Detailed example (cont'd)

Two possibilities:

- Email message has reached Eva's Mail Server
  - MDA will deliver the message to Eva's mailbox
- Email message is forwarded to Eva's Mail server
  - Local Mail Server's MTA transfers the message to Eva's Mail Server (lookup + TCP connection + SMTP session)



Role of the actors involved

MUA (or browser)

- runs on client devices
- offers numerous features to compose/manipulate messages
- transfers messages to (local) Mail Server
- runs the SMTP client component

MSA

- runs on Mail Servers
- receives messages from (trusted) MUAs & transfers them to the MTA
- runs the SMTP server component

MTA

- runs on Mail Servers
- receives messages from the MSA or from other MTAs & transfers them to an MDA or to MTAs (running on recipients Mail Servers)
- runs the SMTP client and server components

MDA

- runs on Mail Servers
- receives messages from MTA & stores them into user mailboxes

SMTP

SMTP is the standard application layer protocol based on a client/server model

Dialog between clients and servers consists of commands sent by the clients and responses sent by the servers

Commands specify the actions requested to transfer mail messages

Syntax of commands: **`NAME`** [optional parameters]

Responses specify the command outcomes (i.e., accepted or refused due to permanent/temporary errors)

Syntax of responses: a three digit reply code, a status code of three numerical fields separated by "." and an explanatory sentence

First sub-code denotes whether the transfer was successful, second the probable source of transfer anomalies, third a precise error condition

SMTP commands/responses: examples

```
EHLO <domain-name>

MAIL FROM: <>
RCPT TO: <>

DATA

RSET
NOOP

QUIT
```

sender/recipient email addresses

message content will follow

server responses

```
250 2.0.0 OK
500 5.5.1 Command unrecognized
```

SMTP session

SMTP is a stateful protocol

An *SMTP* session is a sequence of commands (with a specific order) and responses to transfer mail messages using a single persistent TCP connection

A session is subdivided into three main phases:

1. **Greeting** phase corresponding to the start of an SMTP session
2. **Mail transaction** phase consisting of three steps:
    - specification of the email address of the sender
    - specification of the email address(es) of the recipient(s)
    - specification of the content of the email message
3. **Closing** phase corresponding to the client requesting to close the SMTP session

Multiple email messages can be sent in a single SMTP session (timeouts associated with individual commands)

SMTP commands

EHLO <domain-name> — greetings

MAIL FROM: <>
RCPT TO: <> — mail transaction
DATA

RSET — any phase
NOOP

QUIT — closing

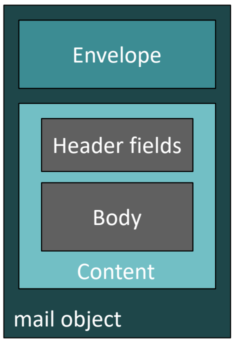
Mail object

SMTP transfers mail objects organized according to the Internet Message Format [RFC 5322 "Internet Message Format" published in 2008]

A mail object consists of

- An envelope that includes the email addresses of the sender (i.e., reverse path for notifications) and recipient(s) (i.e., forward path) to accomplish transfer and delivery

- Content that includes header fields and a body, i.e., the content to be delivered to the recipient organized according to Multipurpose Internet Mail Extension (MIME) format



SMTP commands

```
EHLO <domain-name>

MAIL FROM: <>
RCPT TO: <>

DATA

RSET
NOOP

QUIT
```

envelope

content

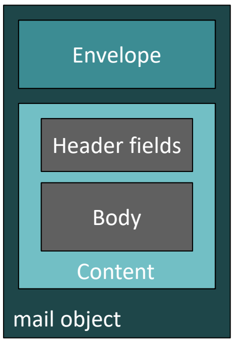
Internet Message Format

The content of a mail object is a sequence of lines terminated by two characters, i.e., carriage-return (**CR**) and line-feed (**LF**)

Special syntax for the header section:

`Field-name:` field value <**CRLF**>

The body is a sequence of characters that follows the header section and is separated from the header section by an empty line (**CRLF**)



Header fields

Header fields are inserted into an email message by:

1. MUA/browser (according to user preferences/specifications)
2. Mail agents that receive and transfer the message

Examples:

Subject:
From:
To:
Cc:
Bcc:
Content-type:
Date:
Message-ID:
Received:
Return-path: